



# Politechnika Warszawska

Wydział Matematyki  
i Nauk Informatycznych

Reprezentacja Wiedzy

## Projekt nr 4: Procesy działań złożonych

Punkt kontrolny 1

Zespół projektowy nr 4:

Wojciech Jasiński (koordynator)

Krzysztof Gryboś

Jarosław Bieniek

Filip Filipkowski

Sebastian Rydz

Jakub Pietrzak

Sandra Adamiec

Warszawa, 26 marca 2026

# 1 Cel projektu

Zadanie polega na zaprojektowaniu dwóch formalnych języków dla klasy *DS4* systemów dynamicznych:

- *języka akcji* — służącego do opisu dziedziny (fluenty, akcje, ich skutki, ograniczenia),
- *języka kwerend* — pozwalającego zadawać pytania o wykonywalność procesów i osiągalność celów.

Klasa *DS4* łączy cechy kilku języków akcji znanych z wykładów. Mechanizmy inercji, niedeterminizmu, ramifikacji i niewykonalności pochodzą z języka *AR* (wykład 2), natomiast pojęcie akcji złożonej nawiązuje do języka *AC* (wykład 5), choć w formie uproszczonej — zamiast pełnego mechanizmu dekompozycji wystarczy warunek rozłączności zbiorów fluentów.

## 2 Język akcji

| Zdanie                                     | Co oznacza                                                                    |
|--------------------------------------------|-------------------------------------------------------------------------------|
| $A \text{ causes } \alpha \text{ if } \pi$ | Akcja $A$ powoduje efekt $\alpha$ , o ile zachodzi warunek $\pi$ .            |
| $A \text{ releases } f \text{ if } \pi$    | Akcja $A$ „zwalnia” fluent — może, ale nie musi się zmienić (niedeterminizm). |
| $\text{impossible } A \text{ if } \pi$     | Akcja $A$ jest niewykonalna, gdy zachodzi $\pi$ .                             |
| $\text{always } \alpha$                    | Formuła $\alpha$ musi być prawdziwa w każdym stanie (ograniczenie).           |
| $\text{noninertial } f$                    | Fluent $f$ nie podlega inercji — może się zmieniać jako skutek uboczny.       |
| $\text{initially } \alpha$                 | Na początku zachodzi $\alpha$ .                                               |
| $\alpha \text{ after } A_1, \dots, A_n$    | Po wykonaniu ciągu akcji zachodzi $\alpha$ (obserwacja).                      |

## 3 Semantyka

### Stany

Stan to przypisanie wartości prawda/fałsz każdemu fluentowi. Nie każdy stan jest dopuszczalny — odrzucamy te, które łamią ograniczenia **always**. Na początku bierzemy te stany dopuszczalne, które dodatkowo spełniają zdania **initially** i obserwacje — to nasze stany początkowe.

### Wykonanie akcji prostej

Żeby obliczyć, co się dzieje po wykonaniu akcji  $A$  w stanie  $\sigma$ :

1. Sprawdzamy, czy  $A$  nie jest niewykonalna (czy nie ma aktywnego **impossible**).
2. Zbieramy efekty — patrzymy, które zdania **causes** mają spełniony warunek wstępny w  $\sigma$ , i łączymy ich efekty.
3. Szukamy stanów dopuszczalnych, w których te efekty zachodzą.
4. Spośród nich wybieramy te, które zmieniły się *jak najmniej* w porównaniu do  $\sigma$  (inercja).  
Fluenty **noninertial** nie liczą się przy tym porównaniu — mogą się zmieniać „za darmo”.

Wynikiem może być kilka stanów (niedeterminizm), jeden stan, albo żaden (niewykonalność).

## Akcje złożone

Akcja złożona to zbiór akcji prostych wykonywanych równocześnie. Żeby taka akcja była wykonalna, każda składowa musi być osobno wykonalna, a pary składowych muszą wpływać na rozłączne zbiory fluentów (żeby efekty się nie gryzły). Wynik obliczamy tak samo jak dla akcji prostej — zbieramy efekty wszystkich składowych i stosujemy inercję. Akcja prosta to po prostu jednoelementowa akcja złożona, np.  $\{T_1\}$ .

## Procesy

Proces to po prostu ciąg akcji złożonych wykonywanych jedna po drugiej. Wynik procesu to zbiór stanów, do których można dojść, wykonując kolejne akcje krok po kroku.

## 4 Język kwerend

Język kwerend pozwala zadawać dwa rodzaje pytań o procesy:

- **Czy proces jest wykonywalny?**
  - **necessary executable**  $P$  — „Czy proces  $P$  da się zawsze wykonać, niezależnie od stanu początkowego i niedeterminizmu?”
  - **possibly executable**  $P$  — „Czy istnieje choć jedna ścieżka, w której da się wykonać cały proces  $P$ ?”
- **Czy po procesie osiągamy cel?**
  - **necessary**  $\gamma$  **after**  $P$  — „Czy po wykonaniu procesu  $P$  *zawsze* zachodzi  $\gamma$ ?”
  - **possibly**  $\gamma$  **after**  $P$  — „Czy *istnieje* ścieżka wykonania  $P$ , po której zachodzi  $\gamma$ ?”

Słowa **necessary** i **possibly** to kwantyfikatory — **necessary** oznacza „dla każdego możliwego przebiegu”, a **possibly** oznacza „istnieje taki przebieg”.

## 5 Przykłady

### 5.1 Dwa przełączniki

Scenariusz oparty na przykładzie 2.4 z wykładu 2, rozszerzony o akcję *Reset* ilustrującą warunek rozłączności akcji złożonych.

Rozważmy system z dwoma przełącznikami ( $s_1, s_2$ ) i światłem.

**Sygatura:**  $\mathcal{F} = \{s_1, s_2, light\}$ ,  $\mathcal{A} = \{T_1, T_2, Reset\}$ .

**Dziedzina  $D$ :**

**noninertial**  $light$   
**initially**  $s_1 \wedge s_2$   
**always**  $light \leftrightarrow (s_1 \leftrightarrow s_2)$   
 $T_1$  **causes**  $\neg s_1$  **if**  $s_1$        $T_1$  **causes**  $s_1$  **if**  $\neg s_1$   
 $T_2$  **causes**  $\neg s_2$  **if**  $s_2$        $T_2$  **causes**  $s_2$  **if**  $\neg s_2$   
 $Reset$  **causes**  $s_1$        $Reset$  **causes**  $s_2$

Światło jest nieinercyjne i jego wartość wynika z ograniczenia: świeci się wtedy i tylko wtedy, gdy oba przełączniki mają tę samą wartość. Na początku oba są włączone, więc światło świeci. *Reset* ustawia oba przełączniki na pozycję „włączone”.

### Stany dopuszczalne:

Mamy 3 fluenty binarne, więc potencjalnie  $2^3 = 8$  stanów, ale ograniczenie **always** przepuszcza tylko 4:

$$\sigma_0 = \{s_1, s_2, l\}, \quad \sigma_1 = \{\neg s_1, \neg s_2, l\}, \quad \sigma_2 = \{s_1, \neg s_2, \neg l\}, \quad \sigma_3 = \{\neg s_1, s_2, \neg l\}$$

Stan początkowy to  $\sigma_0$  (bo **initially**  $s_1 \wedge s_2$ ).

### Co się dzieje po $T_1$ ?

$T_1$  przełącza  $s_1$ . W  $\sigma_0$  mamy  $s_1$  prawdziwe, więc efekt to  $\neg s_1$ . Stany dopuszczalne z  $\neg s_1$  to  $\sigma_1$  i  $\sigma_3$ . Ale  $\sigma_1$  zmienia i  $s_1$ , i  $s_2$  (a  $T_1$  ruszał tylko  $s_1$ ), natomiast  $\sigma_3$  zmienia tylko  $s_1$  — to mniej zmian. Inercja każe wybrać  $\sigma_3$  (*light*, jako nieinercjalny, nie liczy się w porównaniu).

Wynik: po  $T_1$  jesteśmy w  $\sigma_3$ , światło zgasło.

### Kwerendy.

- **necessary  $\neg light$  after** ( $\{T_1\}$ ) — po przełączeniu  $T_1$  światło jest zgaszone?  
**TAK.** Jedyne stany po  $T_1$  to  $\sigma_3$  (patrz wyżej).
- **necessary  $light$  after** ( $\{T_1, T_2\}$ ) — akcja złożona:  $T_1$  rusza  $s_1$ ,  $T_2$  rusza  $s_2$ , zbiory rozłączne, więc akcja jest wykonalna. Efekty:  $\neg s_1 \wedge \neg s_2$ , jedyny pasujący stan to  $\sigma_1$  — światło świeci.  
**TAK.**
- **necessary  $light$  after**  $P$  dla procesu  $P = (\{T_1, T_2\}, \{Reset\})$  — krok 1:  $\{T_1, T_2\}$  w  $\sigma_0 \Rightarrow \sigma_1$ ; krok 2:  $Reset$  w  $\sigma_1 \Rightarrow \sigma_0$ . Wróciliśmy do  $\sigma_0$ , światło świeci.  
**TAK.**
- $\{T_1, Reset\}$  —  $T_1$  wpływa na  $s_1$ ,  $Reset$  wpływa na  $\{s_1, s_2\}$ . Zbiory nie są rozłączne (oba ruszają  $s_1$ ), więc akcja złożona jest **niewykonalna** w każdym stanie.

## 5.2 YSP z rosyjską ruletką

Scenariusz oparty na przykładzie 2.2 z wykładu 2 (rosyjski indyk), rozszerzony o zdania **releases** i **impossible**, częściowy opis stanu początkowego oraz procesy wielokrokowe.

**Sygnatura:**  $\mathcal{F} = \{loaded, alive\}$ ,  $\mathcal{A} = \{Load, Shoot, Spin\}$ .

**Dziedzina  $D$ :**

|                                                                 |                                        |
|-----------------------------------------------------------------|----------------------------------------|
| <b>initially</b> <i>alive</i>                                   | (stan broni nieznany — opis częściowy) |
| <i>Load</i> <b>causes</b> <i>loaded</i>                         |                                        |
| <b>impossible</b> <i>Load</i> <b>if</b> <i>loaded</i>           | (nie można ładować naładowanej broni)  |
| <i>Shoot</i> <b>causes</b> $\neg loaded$                        |                                        |
| <i>Shoot</i> <b>causes</b> $\neg alive$ <b>if</b> <i>loaded</i> |                                        |
| <i>Spin</i> <b>releases</b> <i>loaded</i>                       | (kręcenie bębenkiem — niedeterminizm)  |

### Stany dopuszczalne:

Brak **always**, więc wszystkie cztery stany są dopuszczalne:

$$\begin{aligned} \sigma_0 &= \{loaded, alive\}, & \sigma_1 &= \{loaded, \neg alive\}, \\ \sigma_2 &= \{\neg loaded, alive\}, & \sigma_3 &= \{\neg loaded, \neg alive\}. \end{aligned}$$

Stany początkowe (spełniające **initially** *alive*):  $\Sigma_0 = \{\sigma_0, \sigma_2\}$ .

### Kwerendy dotyczące wykonywalności.

Rozważamy proces  $P = (\{Load\}, \{Spin\}, \{Shoot\})$ .

Z  $\sigma_0$ : *Load* jest *niewykonalne* (broń już załadowana) —  $Res(Load, \sigma_0) = \emptyset$ . Z  $\sigma_2$ : *Load* jest wykonalne:  $\sigma_2 \rightarrow \sigma_0$ .

- **necessary executable**  $P$  — czy  $P$  da się zawsze wykonać?  
**NIE.** Z  $\sigma_0 \in \Sigma_0$  już pierwszy krok jest niewykonalny.
- **possibly executable**  $P$  — czy istnieje ścieżka wykonania  $P$ ?  
**TAK.** Z  $\sigma_2$ :  $Load \rightarrow \sigma_0$ , potem dalej.

#### Kwerendy dotyczące wyniku.

Śledzimy pełne wykonania  $P$  startując z  $\sigma_2$ :

1.  $Load$  w  $\sigma_2$ : efekt  $loaded \Rightarrow \sigma_0 = \{loaded, alive\}$ .
2.  $Spin$  w  $\sigma_0$ :  $loaded$  jest zwolniony (**releases**)  $\Rightarrow Res(Spin, \sigma_0) = \{\sigma_0, \sigma_2\}$  (niedeterminizm).
3.  $Shoot$  po każdej gałęzi:
  - ze  $\sigma_0$  ( $loaded$ ): efekty  $\neg loaded \wedge \neg alive \Rightarrow \sigma_3$ .
  - ze  $\sigma_2$  ( $\neg loaded$ ): **causes** daje  $\neg loaded$  (już spełnione), warunek  $loaded$  w **if** fałszywy  $\Rightarrow$  brak zmiany  $\Rightarrow \sigma_2$ .

Zbiór stanów końcowych pełnych wykonań  $P$ :  $\{\sigma_3, \sigma_2\}$ .

- **necessary  $\neg alive$  after**  $P$  — czy indyk *zawsze* ginie?  
**NIE.** Gałąź przez  $\sigma_2$  kończy się w  $\sigma_2$ , gdzie *alive*.
- **possibly  $\neg alive$  after**  $P$  — czy istnieje ścieżka kończąca się śmiercią?  
**TAK.** Gałąź przez  $\sigma_0$  kończy się w  $\sigma_3$ .
- **possibly *alive* after**  $P$  — czy istnieje ścieżka, na której indyk przeżyje?  
**TAK.** Gałąź przez  $\sigma_2$  (po  $Spin$  broń się rozładowała) kończy się w  $\sigma_2$ .

## 6 Pytania

1. **Ramifikacje vs. kwalifikacje.** Rozumiemy, że ograniczenia **always** działają jak w  $AR$  (odrzucaamy stany niedopuszczalne, a potem stosujemy inercję). Czy potrzebujemy też podejścia z kwalifikacjami (jak w  $AQ$ , wykład 3), gdzie kolejność jest odwrotna?
2. **Co znaczy „wpływa na fluent” przy akcjach złożonych?** Warunek rozłączności mówi, że dwie akcje składowe muszą wpływać na różne fluenty. Ale co to dokładnie znaczy „wpływa”? Czy liczymy tylko fluenty z **causes/releases**, czy też te zmienione pośrednio przez ramifikacje?
3. **Opis częściowy stanów wynikowych.** Stan początkowy może być częściowo opisany — to jasne, daje nam zbiór możliwych stanów. Ale w treści jest też mowa o częściowym opisie stanów wynikowych. Czy to realizujemy po prostu przez zdania obserwacyjne ( $\alpha$  **after** ...), czy jest coś więcej?
4. **Gdzie formalnie umieścić akcje złożone?** W  $AR$  funkcja  $Res$  działa tylko na akcjach prostych. Proponujemy rozszerzyć ją do  $Res: 2^A \times \Sigma \rightarrow 2^\Sigma$  (analogicznie do  $AC$ , wykład 5). Czy to prawidłowe podejście?